

대한상공회의소

# AXI\_SPI & AXI\_I2C

발표자 | 최은수

# | 목차

Chapter 1. **Introduction**

---

Chapter 2. **FSM & Simulation**

---

Chapter 3. **UVM**

---

Chapter 4. **Trouble shooting**

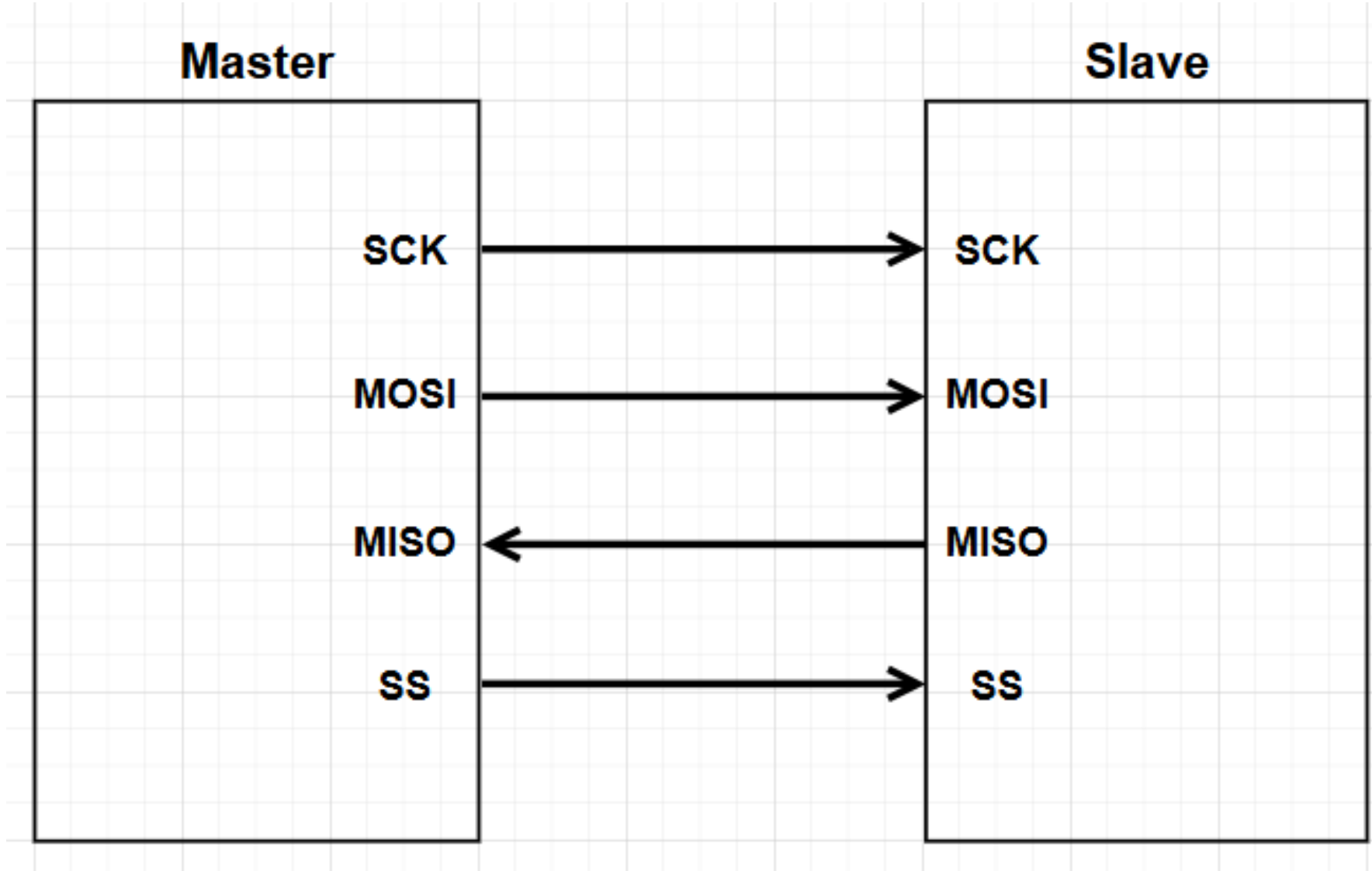
---

Chapter 5. **Demo video**

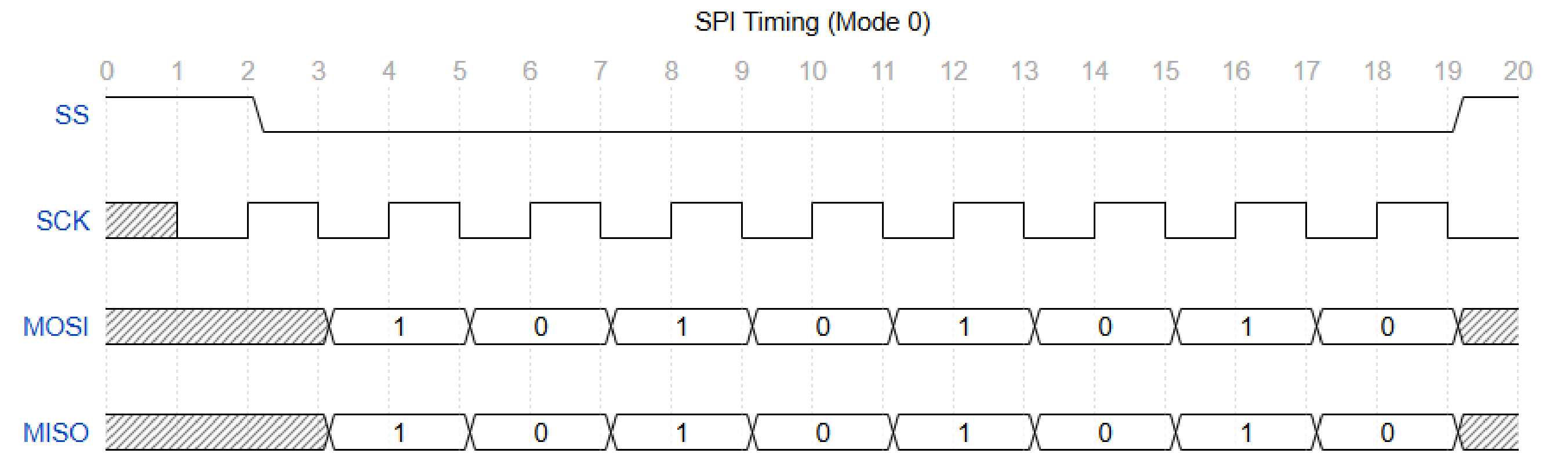
---

# | Introduction of AXI\_SPI

# Introduction of AXI\_SPI

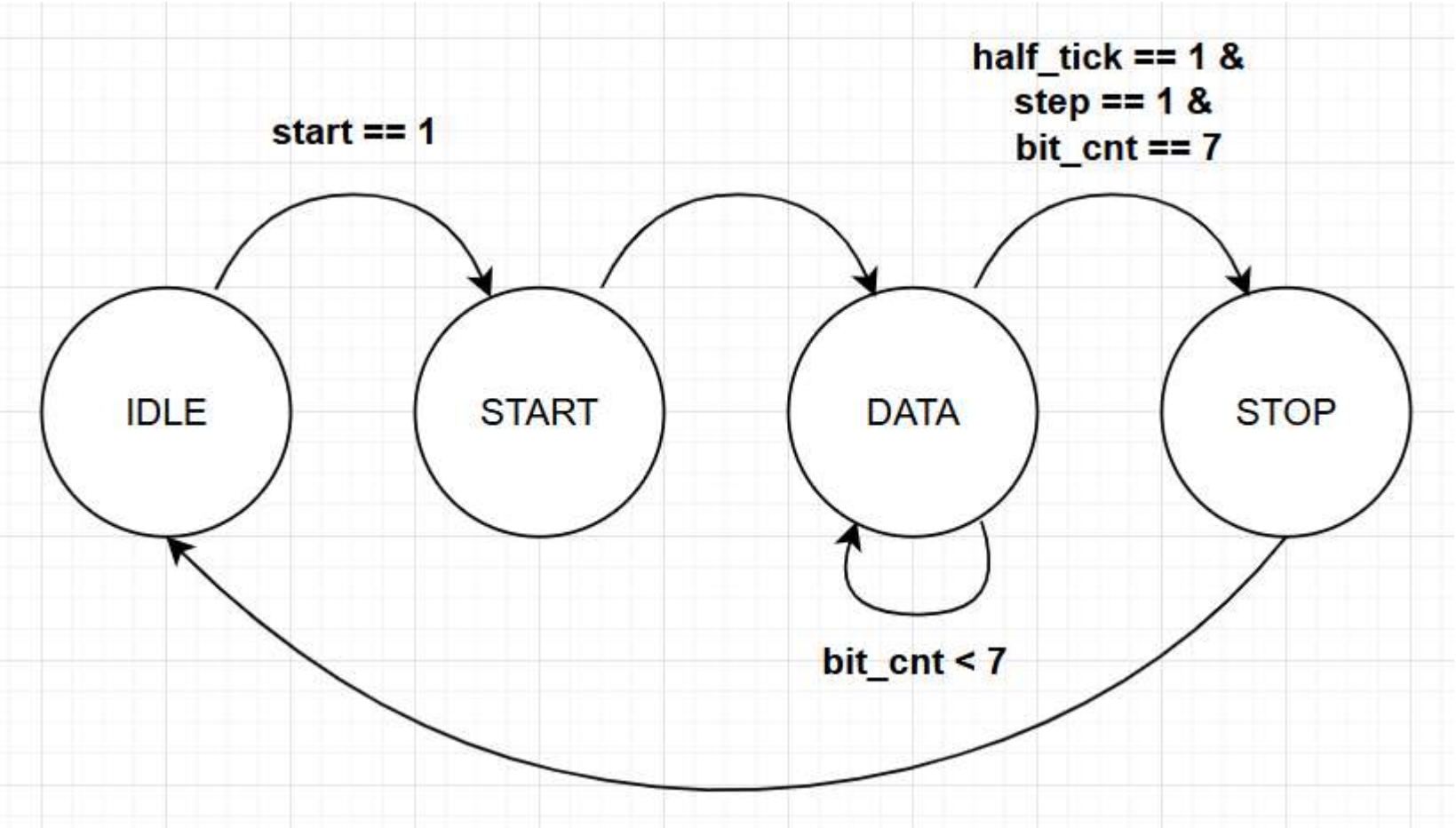


|    | SPI                                                                                 |
|----|-------------------------------------------------------------------------------------|
| 특징 | Master-Slave 기반의 1:N 구조<br>클럭(SCK)에 맞춰 데이터를 주고받는 동기식 직렬통신<br>송신과 수신이 동시에 일어나는 이중 통신 |
| 장점 | I2C나 UART 방식에 비해 데이터 전송 속도가 빠름<br>하드웨어 통신 프로토콜 구조가 단순하고 직관적임                        |
| 단점 | 통신에 필요한 기본 핀(Wire) 수가 많음<br>수신 측의 정상 수신 여부를 확인하는 응답(ACK/NACK) 메커니즘이 없음              |



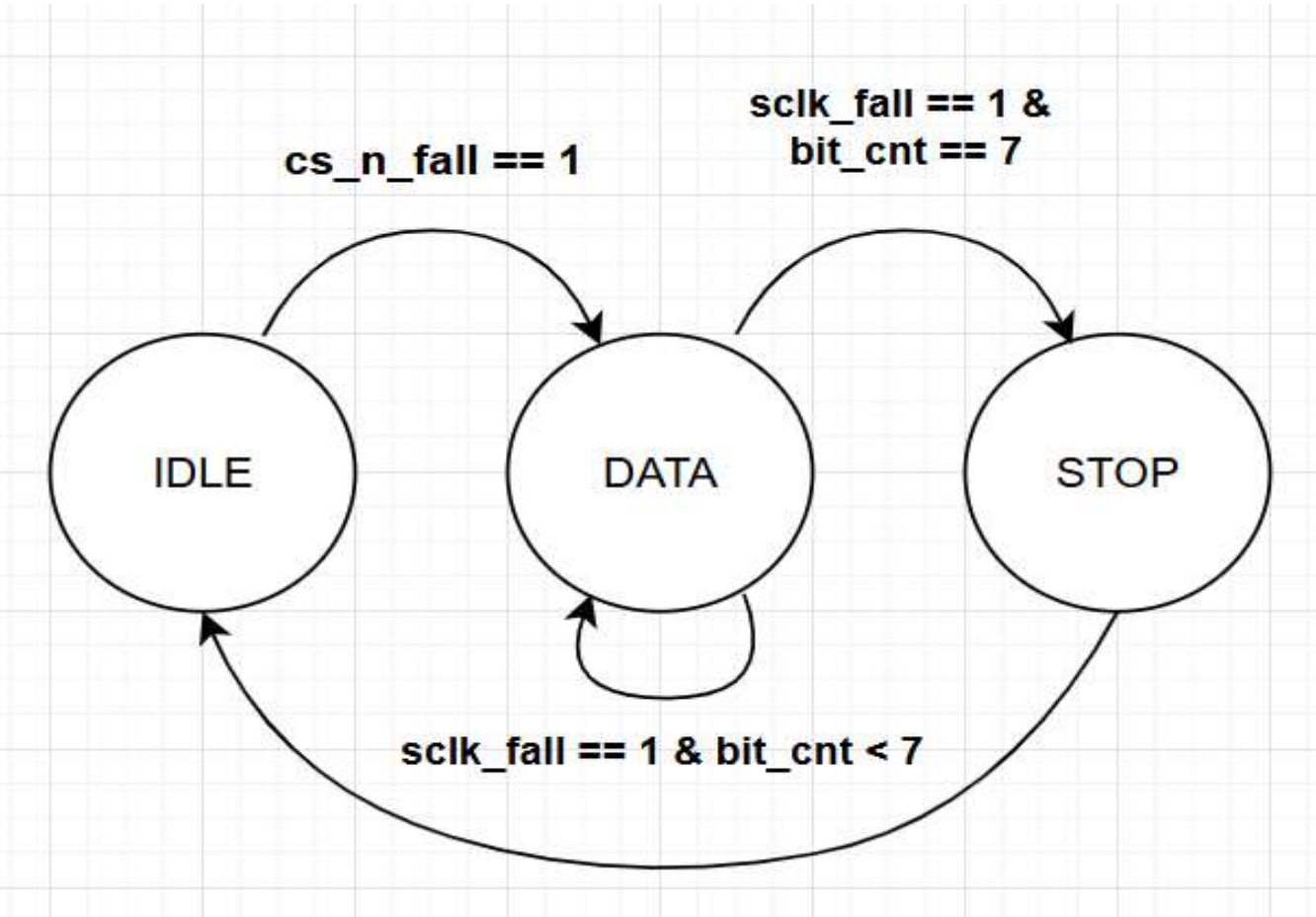
# | FSM & SIMULATION

# FSM



## SPI Master

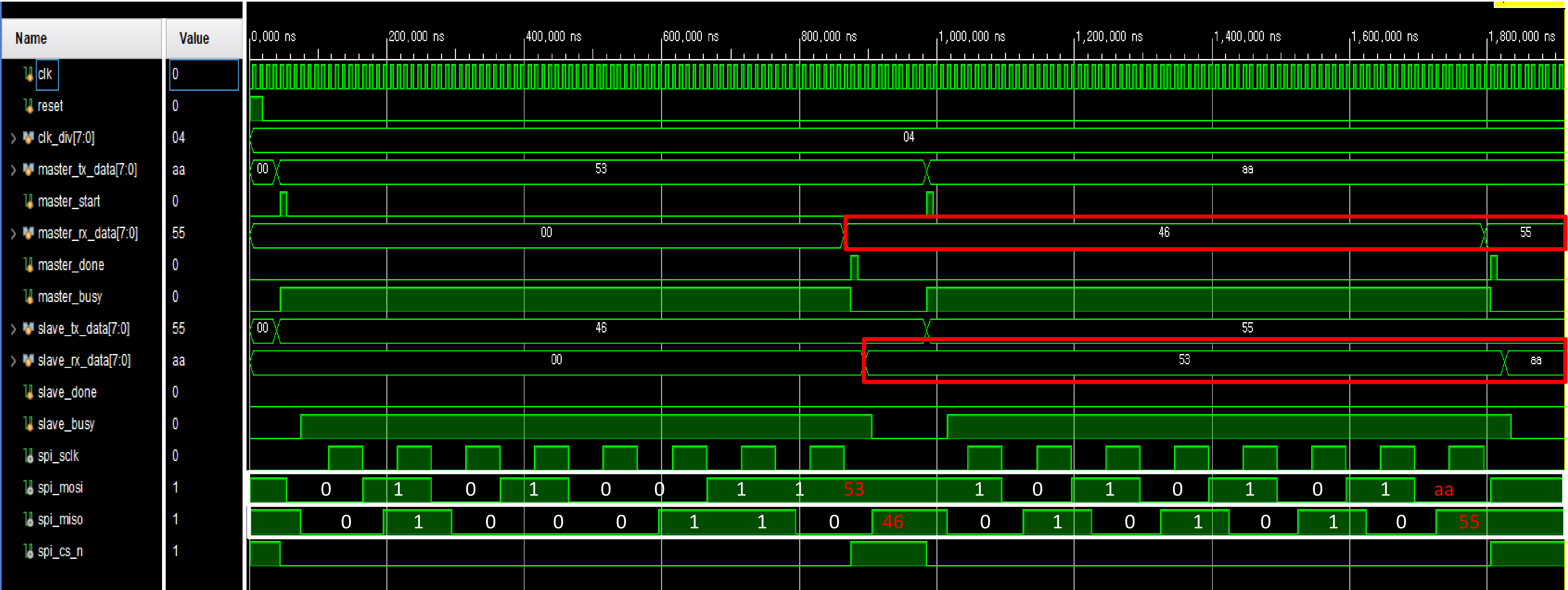
- IDLE -> START -> DATA -> STOP



## SPI Slave

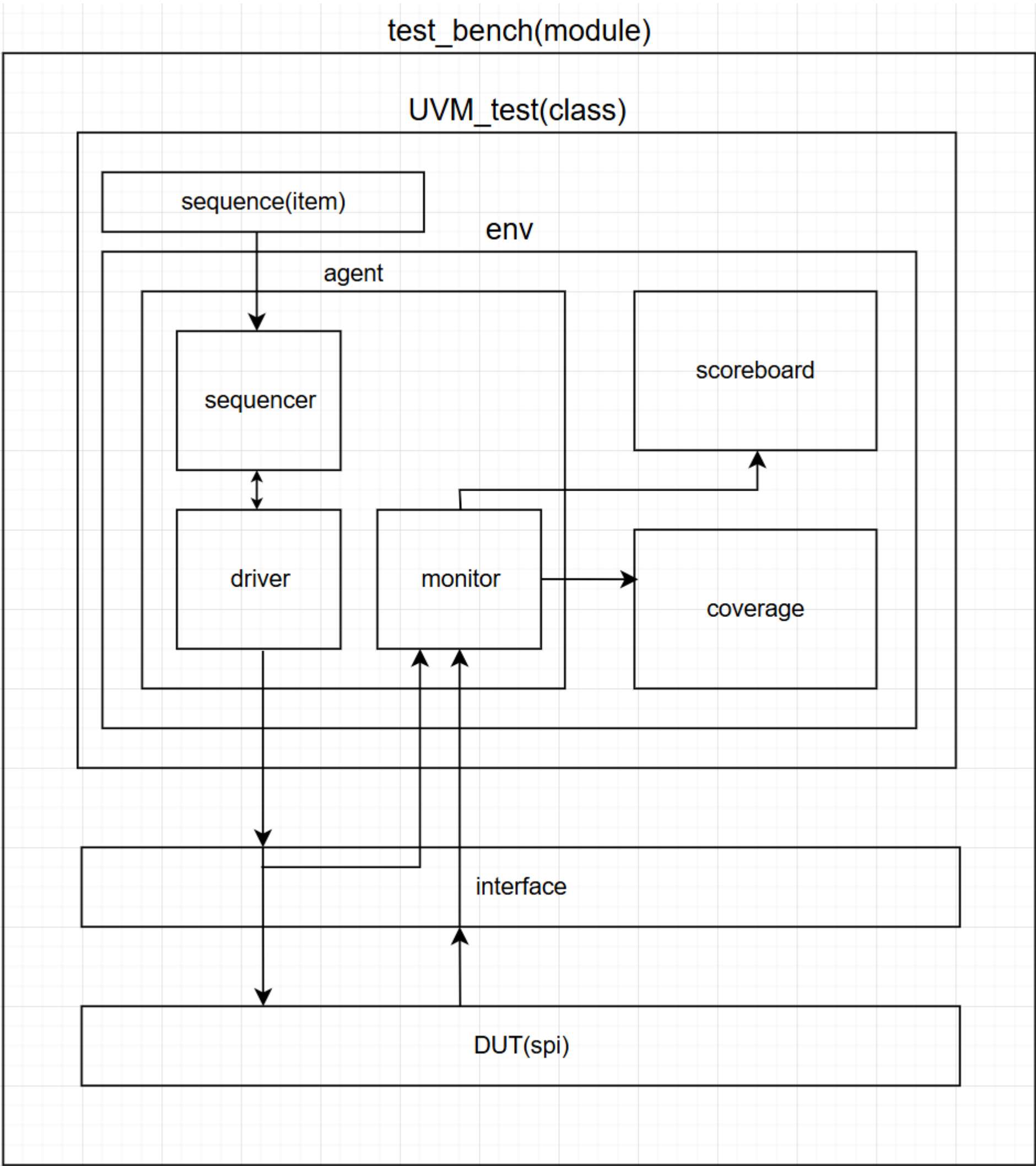
- IDLE -> DATA -> STOP

# SIMULATION



| UVM

# UVM - structure



|                   |                        |    |      |
|-------------------|------------------------|----|------|
| uvm_test_top      | i2c_rand_test          | -  | @342 |
| env               | i2c_env                | -  | @355 |
| agt               | i2c_agent              | -  | @364 |
| drv               | i2c_driver             | -  | @411 |
| rsp_port          | uvm_analysis_port      | -  | @430 |
| seq_item_port     | uvm_seq_item_pull_port | -  | @420 |
| mon               | i2c_monitor            | -  | @402 |
| ap                | uvm_analysis_port      | -  | @578 |
| sqr               | uvm_sequencer          | -  | @440 |
| rsp_export        | uvm_analysis_export    | -  | @449 |
| seq_item_export   | uvm_seq_item_pull_imp  | -  | @567 |
| arbitration_queue | array                  | 0  | -    |
| lock_queue        | array                  | 0  | -    |
| num_last_reqs     | integral               | 32 | 'd1  |
| num_last_rsps     | integral               | 32 | 'd1  |
| cov               | i2c_coverage           | -  | @382 |
| analysis_imp      | uvm_analysis_imp       | -  | @391 |
| scb               | i2c_scoreboard         | -  | @373 |
| ap_imp            | uvm_analysis_imp       | -  | @594 |

# UVM - scenario

```
constraint c_m_tx_data_corner {
    m_tx_data dist {
        8'h00 := 1,
        8'hFF := 1,
        8'haa := 1,
        8'h55 := 1,
        [8'h01:8'hFE] :/ 6
    };
}

constraint c_s_tx_data_corner {
    s_tx_data dist {
        8'h00 := 1,
        8'hFF := 1,
        8'haa := 1,
        8'h55 := 1,
        [8'h01:8'hFE] :/ 6
    };
}
```

```
task body();
    spi_seq_item item;
    repeat (num_trans) begin
        item = spi_seq_item::type_id::create("item");
        start_item(item);
        if (!item.randomize()) begin
            `uvm_fatal(get_type_name(), "spi_seq_item randomize() fail!")
        end
        `uvm_info(get_type_name(), item.convert2string(), UVM_MEDIUM)
        finish_item(item);
    end
endtask
```

## SPI seq\_item

- 0x00 -> All zero
- 0xFF -> All one
- 0xAA -> 10101010
- 0x55 -> 01010101
- 0x01 ~ 0xFE -> 랜덤값

## SPI sequencer

- spi\_seq\_item 객체 생성
- start\_item() → Driver에 전달 준비
- randomize() → 제약 기반 랜덤 데이터 생성
- uvm\_info로 로그 출력
- finish\_item() → Driver에 전달 완료
- num\_trans 횟수만큼 반복

# UVM - scenario

```
virtual task run_phase(uvm_phase phase);
    spi_seq_item item;

    sif.drv_cb.m_start <= 1'b0;
    sif.drv_cb.clk_div <= 8'h00;
    sif.drv_cb.m_tx_data <= 8'h00;
    sif.drv_cb.s_tx_data <= 8'h00;

    wait(sif.reset == 1'b0);
    @(sif.drv_cb);

    forever begin
        seq_item_port.get_next_item(item);
        drive_item(item);
        seq_item_port.item_done();
    end
endtask
```

```
virtual task drive_item(spi_seq_item item);
    @(sif.drv_cb);
    sif.drv_cb.clk_div <= item.clk_div;
    sif.drv_cb.m_tx_data <= item.m_tx_data;
    sif.drv_cb.s_tx_data <= item.s_tx_data;

    sif.drv_cb.m_start <= 1'b1;
    @(sif.drv_cb);
    sif.drv_cb.m_start <= 1'b0;

    while (sif.drv_cb.m_done != 1'b1) begin
        @(sif.drv_cb);
    end

    @(sif.drv_cb);
endtask
```

```
task collect_transaction();
    spi_seq_item tx;
    @(sif.mon_cb);
    if (sif.mon_cb.m_done == 1'b1) begin
        tx = spi_seq_item::type_id::create("mon_tx");
        tx.clk_div = sif.mon_cb.clk_div;
        tx.m_tx_data = sif.mon_cb.m_tx_data;
        tx.s_tx_data = sif.mon_cb.s_tx_data;
        repeat(5) @(sif.mon_cb);
        tx.m_rx_data = sif.mon_cb.m_rx_data;
        tx.s_rx_data = sif.mon_cb.s_rx_data;
        `uvm_info(get_type_name(), $sformatf("mon tx: %s",
            tx.convert2string()), UVM_MEDIUM)
        ap.write(tx);
    end
endtask
```

## SPI driver

[run\_phase]

1. m\_start = 0, clk\_div = 0x00 m\_tx\_data = 0x00, s\_tx\_data = 0x00
2. Reset 해제 대기
3. 반복 루프 시작

[drive\_item]

1. clk\_div, m\_tx\_data, s\_tx\_data 인가
2. m\_start = 1 → 1클럭 → m\_start = 0
3. m\_done == 1 될 때까지 대기
4. 1클럭 후 완료

## SPI monitor

1. m\_done == 1 감지 (전송 완료 감지)
2. 5클럭 대기 (데이터 안정화)
3. clk\_div, m\_tx\_data, s\_tx\_data 캡처
4. m\_rx\_data, s\_rx\_data 캡처
5. analysis port를 통해 Scoreboard / Coverage로 전달

# UVM - scenario

```
function void write(spi_seq_item item);
    if ((item.m_rx_data === item.s_tx_data) && (item.s_rx_data === item.m_tx_data)) begin
        pass_cnt++;
        `uvm_info(get_type_name(), $sformatf("Match!! Master RX: 0x%02h, Slave RX: 0x%02h",
            item.m_rx_data, item.s_rx_data), UVM_MEDIUM)
    end else begin
        fail_cnt++;
        `uvm_error(get_type_name(), $sformatf("Mismatch!! Master RX: 0x%02h (Expected: 0x%02h),
            Slave RX: 0x%02h (Expected: 0x%02h)",
            item.m_rx_data, item.s_tx_data, item.s_rx_data, item.m_tx_data))
    end
endfunction
```

```
covergroup spi_cg;
    cp_m_tx_data: coverpoint tx.m_tx_data {
        bins zero = {8'h00};
        bins alt_01 = {8'h55};
        bins alt_10 = {8'haa};
        bins lsb_only = {8'h01};
        bins msb_only = {8'h80};
        bins low = {[8'h00:8'h3f]};
        bins mid = {[8'h40:8'hbf]};
        bins high = {[8'hc0:8'hff]};
    }
    cp_s_tx_data: coverpoint tx.s_tx_data {
        bins zero = {8'h00};
        bins alt_01 = {8'h55};
        bins alt_10 = {8'haa};
        bins lsb_only = {8'h01};
        bins msb_only = {8'h80};
        bins low = {[8'h00:8'h3f]};
        bins mid = {[8'h40:8'hbf]};
        bins high = {[8'hc0:8'hff]};
    }
endgroup
```

## SPI scoreboard

- m\_rx\_data == s\_tx\_data
- s\_rx\_data == m\_tx\_data
- PASS : pass\_cnt++
- FAIL : fail\_cnt++ & uvm\_error 출력

## SPI coverage

- zero (0x00) -> All zero
- alt\_01 (0x55) -> 01010101
- alt\_10 (0xAA) -> 10101010
- lsb\_only (0x01) -> LSB만 1
- Msb\_only (0x80 -> MSB만 1
- low (0x00 ~ 0x3F -> 하위 범위
- mid (0x40 ~ 0xBF -> 중간 범위
- high (0xC0 ~ 0xFF) -> 상위 범위

# UVM – coverage

```
UVM_INFO ./tb/spi_coverage.sv(46) @ 209805000: uvm_test_top.env.cov [spi_coverage] ==== Coverage Summary ====
UVM_INFO ./tb/spi_coverage.sv(47) @ 209805000: uvm_test_top.env.cov [spi_coverage] Overall: 100.0%
UVM_INFO ./tb/spi_coverage.sv(49) @ 209805000: uvm_test_top.env.cov [spi_coverage] m_tx_data : 100.0%
UVM_INFO ./tb/spi_coverage.sv(51) @ 209805000: uvm_test_top.env.cov [spi_coverage] s_tx_data : 100.0%
UVM_INFO ./tb/spi_coverage.sv(53) @ 209805000: uvm_test_top.env.cov [spi_coverage] ==== Coverage Summary ====

UVM_INFO ./tb/spi_scoreboard.sv(38) @ 209805000: uvm_test_top.env.scb [spi_scoreboard]

UVM_INFO ./tb/spi_scoreboard.sv(39) @ 209805000: uvm_test_top.env.scb [spi_scoreboard] ==== Scoreboard Summary ====
UVM_INFO ./tb/spi_scoreboard.sv(40) @ 209805000: uvm_test_top.env.scb [spi_scoreboard] Total transaction: 10000
UVM_INFO ./tb/spi_scoreboard.sv(42) @ 209805000: uvm_test_top.env.scb [spi_scoreboard] PASS: 10000
UVM_INFO ./tb/spi_scoreboard.sv(43) @ 209805000: uvm_test_top.env.scb [spi_scoreboard] FAIL: 0
UVM_INFO ./tb/spi_scoreboard.sv(49) @ 209805000: uvm_test_top.env.scb [spi_scoreboard] TEST PASSED: 10000 all matches detected!
UVM_INFO ./tb/spi_scoreboard.sv(52) @ 209805000: uvm_test_top.env.scb [spi_scoreboard]
```

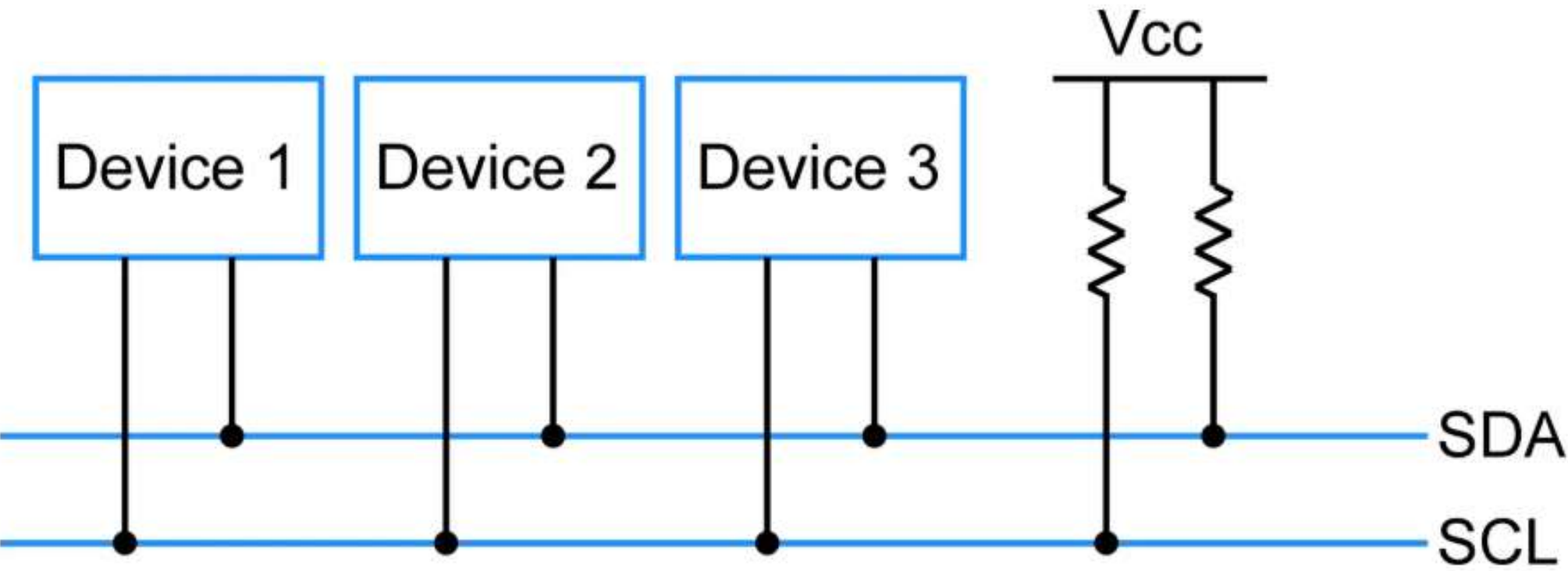
|  | Cover Group Item      | Score   | Instances | U+C | U | C  | X | Goal |
|--|-----------------------|---------|-----------|-----|---|----|---|------|
|  | CG \$unit::i2c_cov... | 100.00% |           | 23  | 0 | 23 | 0 | 100% |
|  | CP cp_m_tx_...        | 100.00% |           | 11  | 0 | 11 | 0 | 100% |
|  | CP cp_op              | 100.00% |           | 4   | 0 | 4  | 0 | 100% |
|  | CP cp_s_tx_d...       | 100.00% |           | 8   | 0 | 8  | 0 | 100% |

|   |            |      |   |   |       |
|---|------------|------|---|---|-------|
| ✓ | addr_base  | User | 1 | 1 | 650   |
| ✓ | addr_read  | User | 1 | 1 | 5681  |
| ✓ | addr_write | User | 1 | 1 | 5657  |
| ✓ | alt_01     | User | 1 | 1 | 339   |
| ✓ | alt_10     | User | 1 | 1 | 341   |
| ✓ | high       | User | 1 | 1 | 769   |
| ✓ | low        | User | 1 | 1 | 37696 |
| ✓ | lsb_only   | User | 1 | 1 | 4     |
| ✓ | mid        | User | 1 | 1 | 1535  |
| ✓ | msb_only   | User | 1 | 1 | 8     |
| ✓ | zero       | User | 1 | 1 | 25327 |

| Status | Bin Name | Type | At Least | Size | Hit Count |
|--------|----------|------|----------|------|-----------|
| ✓      | alt_01   | User | 1        | 1    | 92        |
| ✓      | alt_10   | User | 1        | 1    | 72        |
| ✓      | high     | User | 1        | 1    | 5112      |
| ✓      | low      | User | 1        | 1    | 24884     |
| ✓      | lsb_only | User | 1        | 1    | 96        |
| ✓      | mid      | User | 1        | 1    | 10004     |
| ✓      | msb_only | User | 1        | 1    | 66        |
| ✓      | zero     | User | 1        | 1    | 20052     |

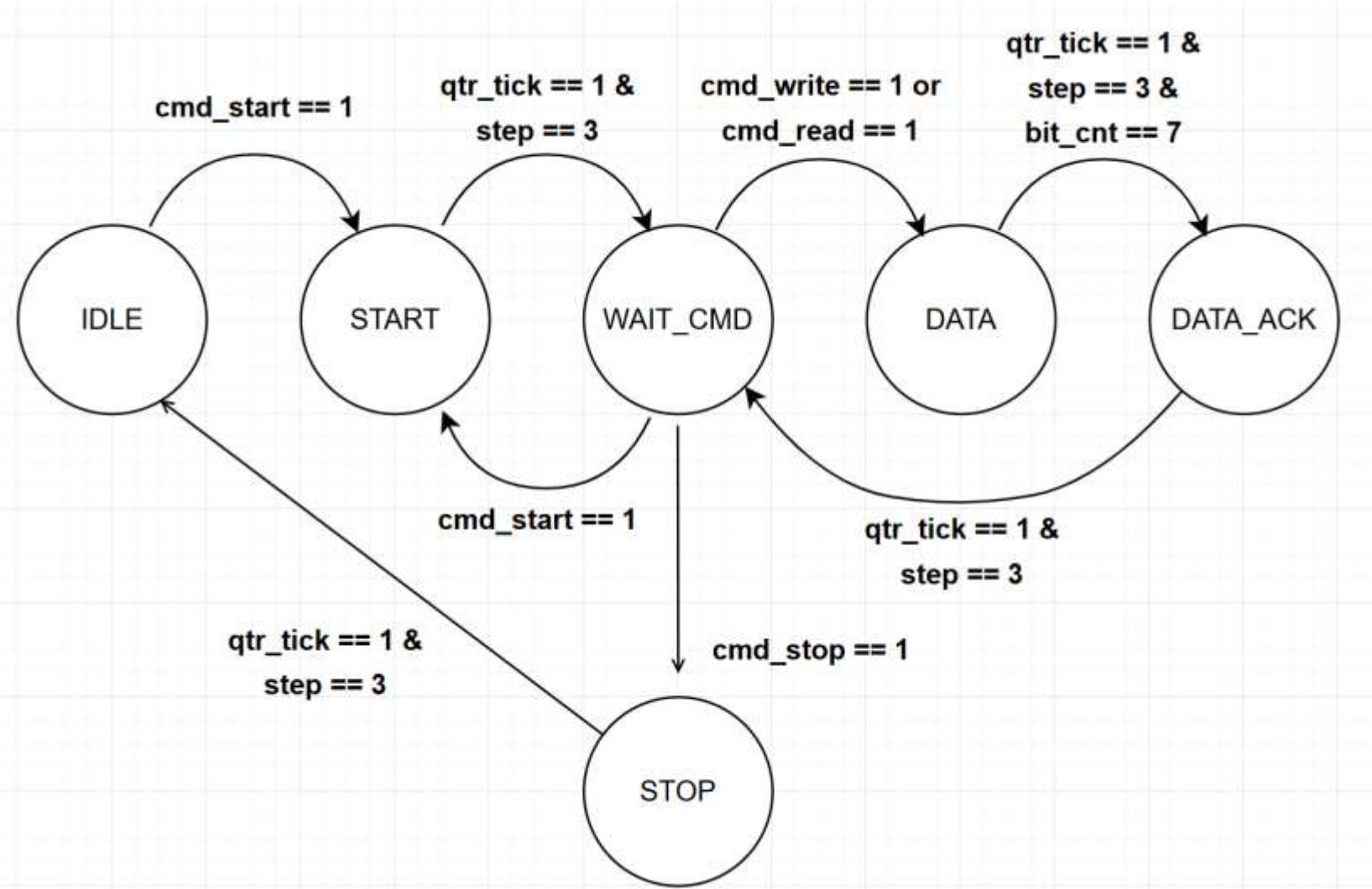
# | Introduction of I2C

# Introduction of I2C



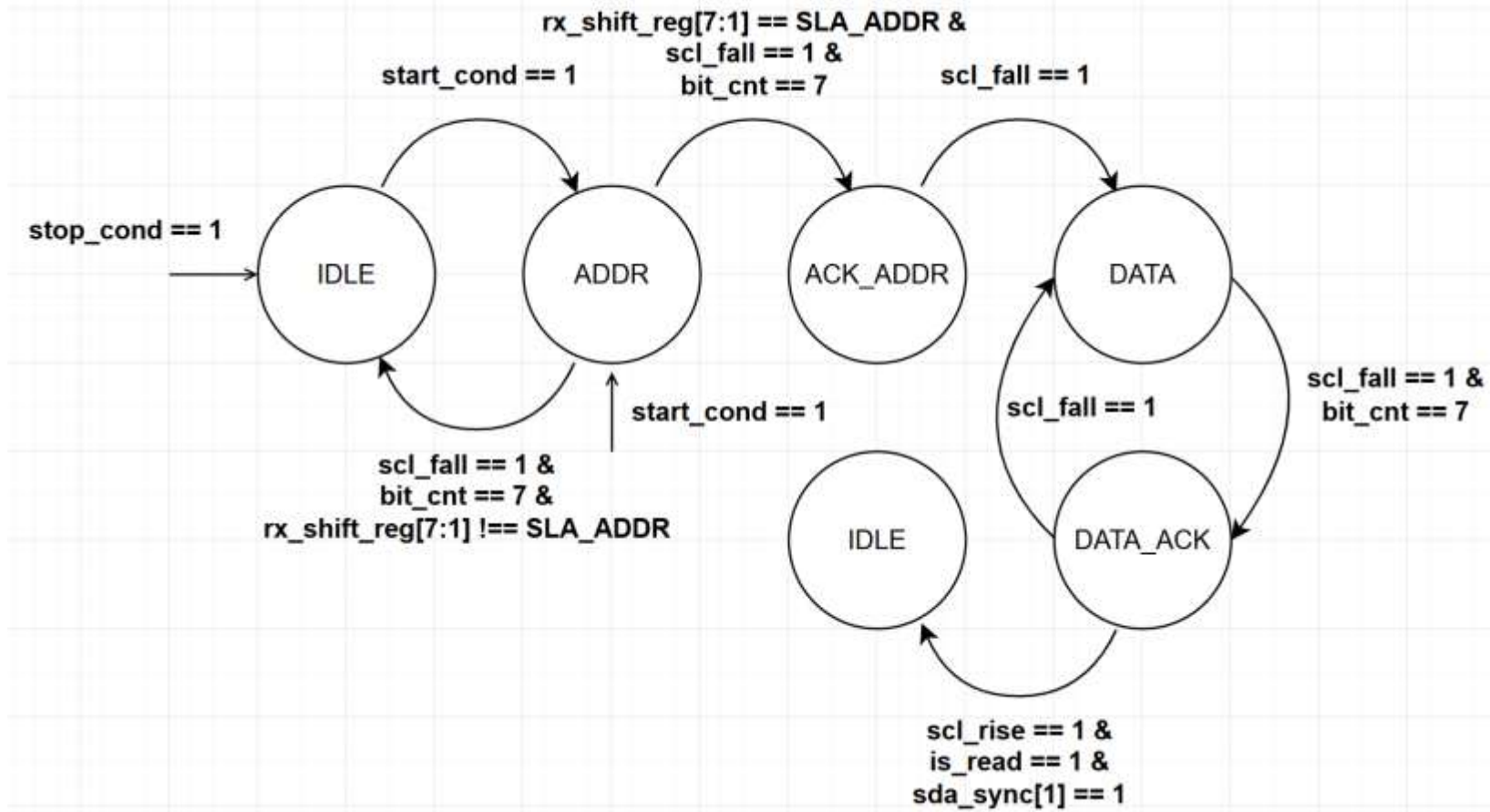
|    | I2C                                                                              |
|----|----------------------------------------------------------------------------------|
| 특징 | 2개의 선만 사용하는 동기식 직렬 통신 방식<br>Open-Drain구조로 Vcc에 연결된 풀업 저항 필수                      |
| 장점 | 선이 2개뿐이라 칩의 Pin 낭비가 적고 저렴함<br>매 바이트마다 ACK/NACK 응답을 주고받아 데이터가 제대로 전달되었는지 즉시 확인 가능 |
| 단점 | SPI 통신에 비해 전송 속도가 현저히 느림<br>데이터를 보내는 동시에 받을 수 없음                                 |

# | FSM & SIMULATION



## I2C Master

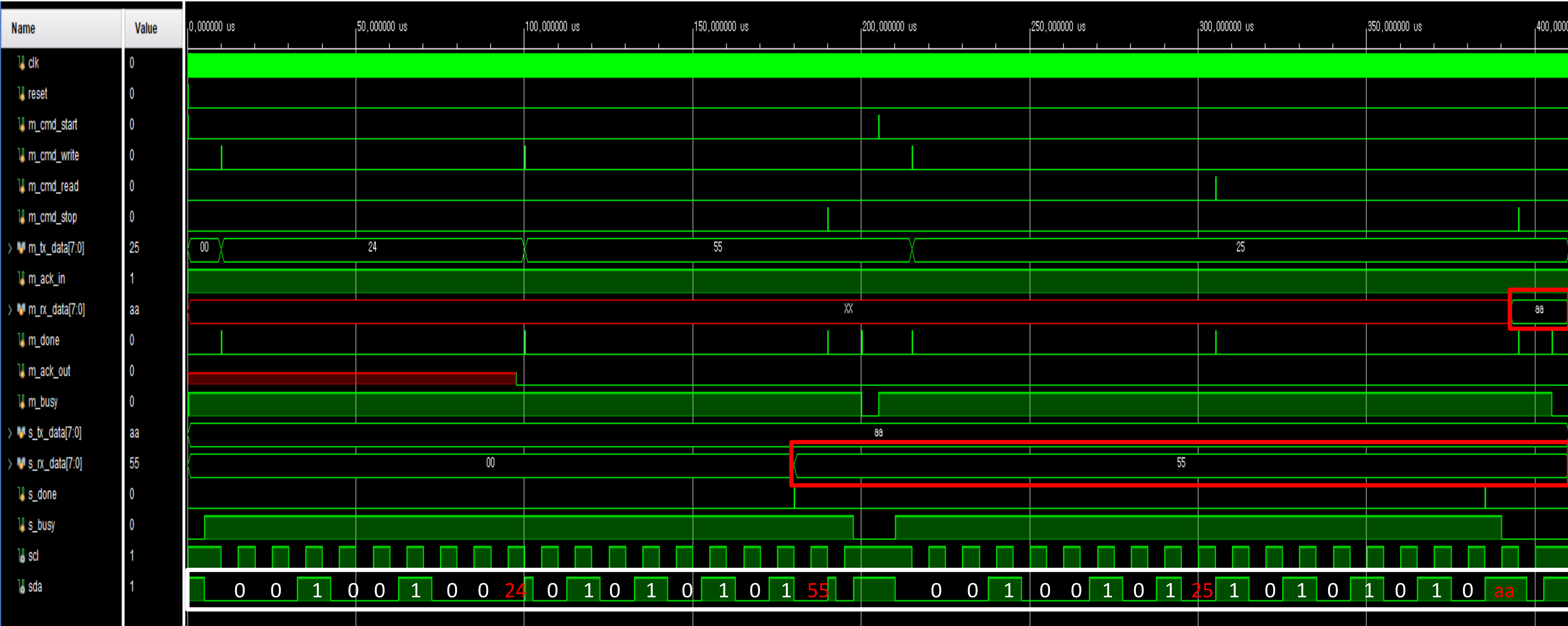
- IDLE -> START -> WAIT\_CMD -> STOP
- IDLE -> START -> WAIT\_CMD -> DATA -> DATA\_ACK -> WAIT\_CMD -> STOP



## I2C Slave

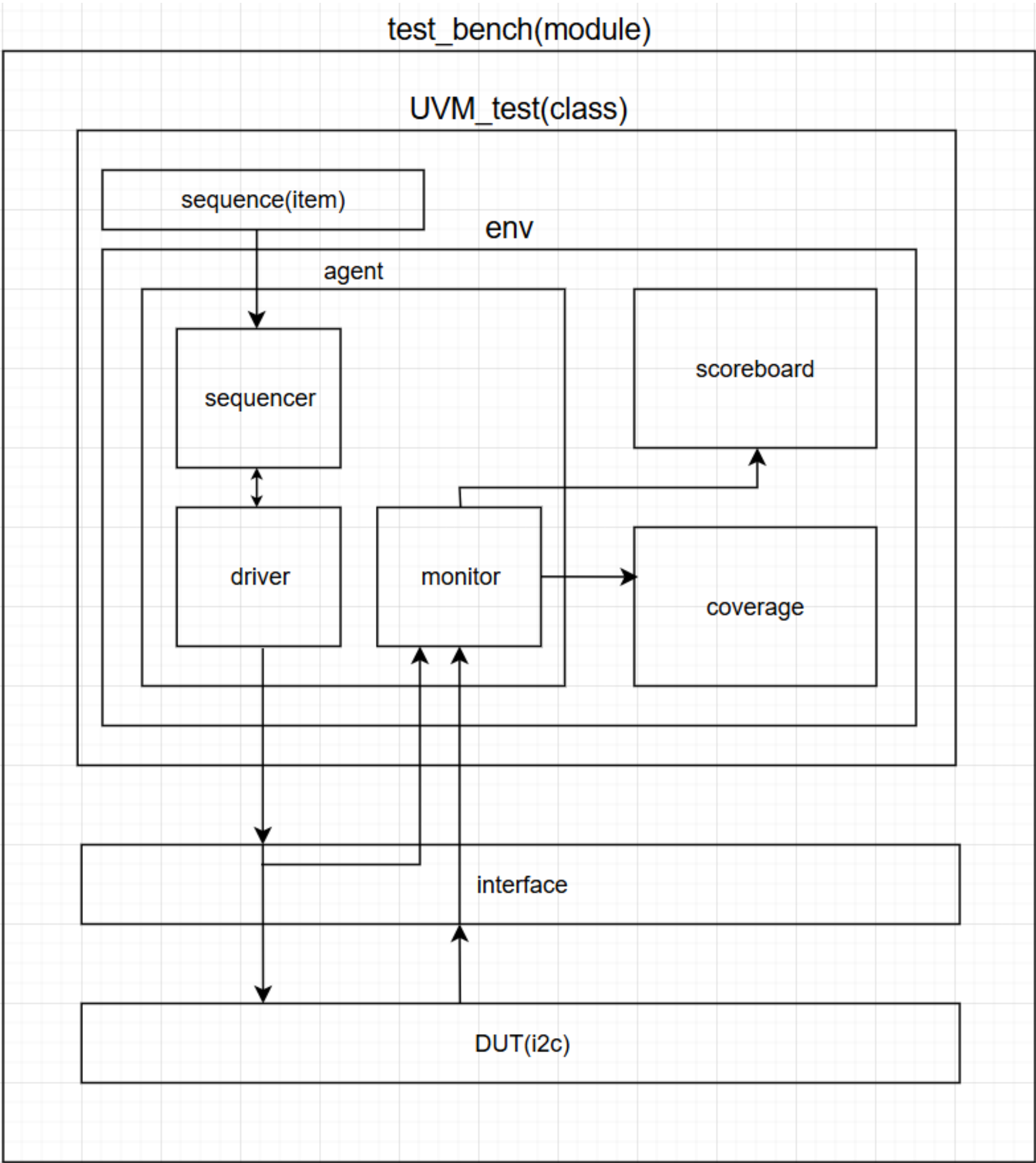
- IDLE -> ADDR -> ACK\_ADDR -> DATA -> DATA\_ACK

# SIMULATION



| UVM

# UVM - structure



| Name              | Type                   | Size | Value |
|-------------------|------------------------|------|-------|
| uvm_test_top      | spi_rand_test          | -    | @342  |
| env               | spi_env                | -    | @355  |
| agt               | spi_agent              | -    | @364  |
| drv               | spi_driver             | -    | @402  |
| rsp_port          | uvm_analysis_port      | -    | @421  |
| seq_item_port     | uvm_seq_item_pull_port | -    | @411  |
| mon               | spi_monitor            | -    | @431  |
| ap                | uvm_analysis_port      | -    | @578  |
| sqr               | uvm_sequencer          | -    | @440  |
| rsp_export        | uvm_analysis_export    | -    | @449  |
| seq_item_export   | uvm_seq_item_pull_imp  | -    | @567  |
| arbitration_queue | array                  | 0    | -     |
| lock_queue        | array                  | 0    | -     |
| num_last_reqs     | integral               | 32   | 'd1   |
| num_last_rsps     | integral               | 32   | 'd1   |
| cov               | spi_coverage           | -    | @382  |
| analysis_imp      | uvm_analysis_imp       | -    | @391  |
| scb               | spi_scoreboard         | -    | @373  |
| ap_imp            | uvm_analysis_imp       | -    | @594  |

# UVM - scenario

```
constraint c_m_tx_data_corner {
    m_tx_data dist {
        8'h00 := 1,
        8'hFF := 1,
        8'haa := 1,
        8'h55 := 1,
        8'h12 := 2,
        8'h24 := 2,
        8'h25 := 2,
        [8'h01:8'hFE] :/ 5
    };
}

constraint c_s_tx_data_corner {
    s_tx_data dist {
        8'h00 := 1,
        8'hFF := 1,
        8'haa := 1,
        8'h55 := 1,
        [8'h01:8'hFE] :/ 6
    };
}
```

```
is_read = $urandom_range(0, 1);
rand_slave_data = $urandom();

req = i2c_seq_item::type_id::create("req");
start_item(req);
req.op = I2C_START;
req.s_tx_data = rand_slave_data;
finish_item(req);

req = i2c_seq_item::type_id::create("req");
start_item(req);
req.op = I2C_WRITE;
if (is_read) begin
    req.m_tx_data = 8'h25;
end else begin
    req.m_tx_data = 8'h24;
end
req.s_tx_data = rand_slave_data;
finish_item(req);

req = i2c_seq_item::type_id::create("req");
start_item(req);
if (is_read) begin
    req.op = I2C_READ;
    req.randomize();
    req.m_ack_in = 1'b1;
end else begin
    req.op = I2C_WRITE;
    req.randomize();
end
req.s_tx_data = rand_slave_data;
finish_item(req);

req = i2c_seq_item::type_id::create("req");
start_item(req);
req.op = I2C_STOP;
req.s_tx_data = rand_slave_data;
finish_item(req);
```

## I2C seq\_item

### [ m\_tx\_data constraint ]

- 0x00 → All zero
- 0xFF → All one
- 0xAA → 10101010
- 0x55 → 01010101
- 0x12 → Slave 기본 주소
- 0x24 → 주소 + Write bit
- 0x25 → 주소 + Read bit
- 0x01~0xFE → 랜덤값

### [ s\_tx\_data constraint ]

- 0x00 → All zero
- 0xFF → All one
- 0xAA → 10101010
- 0x55 → 01010101
- 0x01~0xFE → 랜덤값

## I2C sequencer

- START item 생성
- WRITE item 생성 (주소 프레임)
  - is\_read=1 → m\_tx\_data = 0x25 (주소+R)
  - is\_read=0 → m\_tx\_data = 0x24 (주소+W)
- READ / WRITE item 생성 (데이터 프레임)
  - is\_read=1 → READ op, m\_ack\_in = 1'b1 (NACK)
  - is\_read=0 → WRITE op, randomize()
- STOP item 생성

# UVM - scenario

```

virtual task drive_item(i2c_seq_item item);
  @(iif.drv_cb);
  iif.drv_cb.s_tx_data <= item.s_tx_data;

  case (item.op)
    I2C_START: begin
      iif.drv_cb.m_cmd_start <= 1'b1;
      @(iif.drv_cb);
      iif.drv_cb.m_cmd_start <= 1'b0;
    end
    I2C_WRITE: begin
      iif.drv_cb.m_tx_data <= item.m_tx_data;
      iif.drv_cb.m_cmd_write <= 1'b1;
      @(iif.drv_cb);
      iif.drv_cb.m_cmd_write <= 1'b0;
    end
    I2C_READ: begin
      iif.drv_cb.m_ack_in <= item.m_ack_in;
      iif.drv_cb.m_cmd_read <= 1'b1;
      @(iif.drv_cb);
      iif.drv_cb.m_cmd_read <= 1'b0;
    end
    I2C_STOP: begin
      iif.drv_cb.m_cmd_stop <= 1'b1;
      @(iif.drv_cb);
      iif.drv_cb.m_cmd_stop <= 1'b0;
    end
  endcase

  while (iif.drv_cb.m_done !== 1'b1) begin
    @(iif.drv_cb);
  end

  @(iif.drv_cb);
endtask

```

```

task collect_transaction();
  i2c_seq_item tx;
  tx = i2c_seq_item::type_id::create("mon_tx");

  forever begin
    @(iif.mon_cb);
    if (iif.mon_cb.m_cmd_start) begin
      tx.op = I2C_START;
      break;
    end
    else if (iif.mon_cb.m_cmd_write) begin
      tx.op = I2C_WRITE;
      tx.m_tx_data = iif.mon_cb.m_tx_data;
      tx.s_tx_data = iif.mon_cb.s_tx_data;
      break;
    end
    else if (iif.mon_cb.m_cmd_read) begin
      tx.op = I2C_READ;
      tx.m_ack_in = iif.mon_cb.m_ack_in;
      tx.s_tx_data = iif.mon_cb.s_tx_data;
      break;
    end
    else if (iif.mon_cb.m_cmd_stop) begin
      tx.op = I2C_STOP;
      break;
    end
  end

  while (iif.mon_cb.m_done !== 1'b1) begin
    @(iif.mon_cb);
  end

  tx.m_rx_data = iif.mon_cb.m_rx_data;
  tx.s_rx_data = iif.mon_cb.s_rx_data;
  tx.m_ack_out = iif.mon_cb.m_ack_out;

  `uvm_info(get_type_name(), $sformatf("mon tx: %s",
    tx.convert2string()), UVM_MEDIUM)
  ap.write(tx);

  @(iif.mon_cb);
endtask

```

## I2C driver

[ drive\_item ]

- I2C\_START → m\_cmd\_start 1클럭 펄스 구동
- I2C\_WRITE → m\_tx\_data 세팅 후 m\_cmd\_write 펄스
- I2C\_READ → m\_ack\_in 세팅 후 m\_cmd\_read 펄스
- I2C\_STOP → m\_cmd\_stop 1클럭 펄스 구동

## I2C monitor

[ collect\_transaction ]

- cmd 신호를 클럭마다 감시
- m\_cmd\_start 감지 → op = I2C\_START
- m\_cmd\_write 감지 → op = I2C\_WRITE
  - m\_tx\_data, s\_tx\_data 캡처
- m\_cmd\_read 감지 → op = I2C\_READ
  - m\_ack\_in, s\_tx\_data 캡처
- m\_cmd\_stop 감지 → op = I2C\_STOP
- m\_done = 1'b1 까지 대기 (Driver와 동일 타이밍)
- 결과값 캡처
- m\_rx\_data, s\_rx\_data, m\_ack\_out
- ap.write(tx) → Scoreboard & Coverage 동시 전달

# UVM – scenario

```
covergroup i2c_cg;
  cp_op: coverpoint tx.op {
    bins op_start = {I2C_START};
    bins op_write = {I2C_WRITE};
    bins op_read = {I2C_READ};
    bins op_stop = {I2C_STOP};
  }
  cp_m_tx_data: coverpoint tx.m_tx_data {
    bins addr_base = {8'h12};
    bins addr_write = {8'h24};
    bins addr_read = {8'h25};

    bins zero = {8'h00};
    bins alt_01 = {8'h55};
    bins alt_10 = {8'haa};
    bins lsb_only = {8'h01};
    bins msb_only = {8'h80};
    bins low = {[8'h00:8'h3f]};
    bins mid = {[8'h40:8'hbf]};
    bins high = {[8'hc0:8'hff]};
  }
  cp_s_tx_data: coverpoint tx.s_tx_data {
    bins zero = {8'h00};
    bins alt_01 = {8'h55};
    bins alt_10 = {8'haa};
    bins lsb_only = {8'h01};
    bins msb_only = {8'h80};
    bins low = {[8'h00:8'h3f]};
    bins mid = {[8'h40:8'hbf]};
    bins high = {[8'hc0:8'hff]};
  }
}
```

```
function void write(i2c_seq_item item);

  if (item.op == I2C_START) begin
    is_addr_phase = 1'b1;
  end
  else if (item.op == I2C_WRITE) begin
    if (is_addr_phase == 1'b1) begin
      `uvm_info(get_type_name(), $sformatf("[ADDR PHASE] Master sent
        Addr: 0x%02h (Check skipped)", item.m_tx_data), UVM_HIGH)
      is_addr_phase = 1'b0;
    end
    else begin
      if (item.m_tx_data === item.s_rx_data) begin
        pass_cnt++;
        `uvm_info(get_type_name(), $sformatf("[WRITE PASS] Master TX: 0x%02h
          == Slave RX: 0x%02h", item.m_tx_data, item.s_rx_data), UVM_HIGH)
      end else begin
        fail_cnt++;
        `uvm_error(get_type_name(), $sformatf("[WRITE FAIL] Master TX: 0x%02h
          != Slave RX: 0x%02h", item.m_tx_data, item.s_rx_data))
      end
    end
  end
end

else if (item.op == I2C_READ) begin
  is_addr_phase = 1'b0;
  if (item.s_tx_data === item.m_rx_data) begin
    pass_cnt++;
    `uvm_info(get_type_name(), $sformatf("[READ PASS] Slave TX: 0x%02h
      == Master RX: 0x%02h", item.s_tx_data, item.m_rx_data), UVM_HIGH)
  end else begin
    fail_cnt++;
    `uvm_error(get_type_name(), $sformatf("[READ FAIL] Slave TX: 0x%02h
      != Master RX: 0x%02h", item.s_tx_data, item.m_rx_data))
  end
end
end
endfunction
```

## I2C scoreboard

- | I2C_WRITE                         | I2C_READ                       |
|-----------------------------------|--------------------------------|
| • is_addr_phase = 1 → 주소 프레임 skip | • Slave TX == Master RX → PASS |
| • is_addr_phase = 0 → 데이터 비교      | • Slave TX != Master RX → FAIL |
| • Master TX == Slave RX → PASS    |                                |
| • Master TX != Slave RX → FAIL    |                                |

## I2C coverage

- [ cp\_op ]
- START / WRITE / READ / STOP 4가지 동작 전부 실행 여부 확인
- [ cp\_m\_tx\_data ]
- addr\_base (0x12) → Slave 주소
  - addr\_write (0x24) → 주소 + Write
  - addr\_read (0x25) → 주소 + Read- zero (0x00) / alt\_01 (0x55) / alt\_10 (0xAA)
  - lsb\_only (0x01) / msb\_only (0x80)
  - low [0x00:0x3F] / mid [0x40:0xBF] / high [0xC0:0xFF]

# UVM – coverage

```
UVM_INFO ./tb/i2c_coverage.sv(64) @ 2000002000: uvm_test_top.env.cov [i2c_coverage] ==== Coverage Summary ====
UVM_INFO ./tb/i2c_coverage.sv(65) @ 2000002000: uvm_test_top.env.cov [i2c_coverage] Overall: 100.0%
UVM_INFO ./tb/i2c_coverage.sv(67) @ 2000002000: uvm_test_top.env.cov [i2c_coverage] Operation : 100.0%
UVM_INFO ./tb/i2c_coverage.sv(69) @ 2000002000: uvm_test_top.env.cov [i2c_coverage] m_tx_data : 100.0%
UVM_INFO ./tb/i2c_coverage.sv(71) @ 2000002000: uvm_test_top.env.cov [i2c_coverage] s_tx_data : 100.0%
UVM_INFO ./tb/i2c_coverage.sv(77) @ 2000002000: uvm_test_top.env.cov [i2c_coverage] ==== Coverage Summary ====

UVM_INFO ./tb/i2c_scoreboard.sv(64) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard]

UVM_INFO ./tb/i2c_scoreboard.sv(65) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard] ==== Scoreboard Summary ====
UVM_INFO ./tb/i2c_scoreboard.sv(66) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard] Total transaction: 10000
UVM_INFO ./tb/i2c_scoreboard.sv(68) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard] PASS: 10000
UVM_INFO ./tb/i2c_scoreboard.sv(69) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard] FAIL: 0
UVM_INFO ./tb/i2c_scoreboard.sv(75) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard] TEST PASSED: 10000 all matches detected!
UVM_INFO ./tb/i2c_scoreboard.sv(78) @ 2000002000: uvm_test_top.env.scb [i2c_scoreboard]
```

| Cover Group Item                                                                                       | Score                          | Instances | U+C | U | C  | X | Goal |
|--------------------------------------------------------------------------------------------------------|--------------------------------|-----------|-----|---|----|---|------|
|  \$unit::spi_cov... | <div><div></div></div> 100.00% |           | 16  | 0 | 16 | 0 | 100% |
|  cp_m_tx_...        | <div><div></div></div> 100.00% |           | 8   | 0 | 8  | 0 | 100% |
|  cp_s_tx_d...       | <div><div></div></div> 100.00% |           | 8   | 0 | 8  | 0 | 100% |

|  | Status                                                                              | Bin Name | Type | At Least | Size | Hit Count |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------|------|----------|------|-----------|
|                                                                                     |  | alt_01   | User | 1        | 1    | 985       |
|                                                                                     |  | alt_10   | User | 1        | 1    | 997       |
|                                                                                     |  | high     | User | 1        | 1    | 2499      |
|                                                                                     |  | low      | User | 1        | 1    | 2538      |
|                                                                                     |  | lsb_only | User | 1        | 1    | 22        |
|                                                                                     |  | mid      | User | 1        | 1    | 4963      |
|                                                                                     |  | msb_only | User | 1        | 1    | 26        |
|                                                                                     |  | zero     | User | 1        | 1    | 976       |

|  | Status                                                                                | Bin Name | Type | At Least | Size | Hit Count |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------|------|----------|------|-----------|
|                                                                                       |  | alt_01   | User | 1        | 1    | 1041      |
|                                                                                       |  | alt_10   | User | 1        | 1    | 1046      |
|                                                                                       |  | high     | User | 1        | 1    | 2468      |
|                                                                                       |  | low      | User | 1        | 1    | 2440      |
|                                                                                       |  | lsb_only | User | 1        | 1    | 26        |
|                                                                                       |  | mid      | User | 1        | 1    | 5092      |
|                                                                                       |  | msb_only | User | 1        | 1    | 26        |
|                                                                                       |  | zero     | User | 1        | 1    | 1009      |

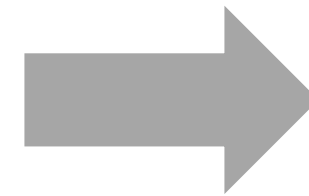
# | Trouble shooting

# Trouble shooting

```
[spi_monitor] mon tx: clk_div: 0x02, MASTER -> tx:0xaa rx:0xeb, SLAVE -> tx:0xd6 rx:0xaa  
[spi_scoreboard] Mismatch!! Master RX: 0xeb (Expected: 0xd6) | Slave RX: 0xaa (Expected: 0xaa)
```

```
rand logic [7:0] clk_div;  
rand logic [7:0] m_tx_data;  
rand logic [7:0] s_tx_data;  
logic [7:0] m_rx_data;  
logic [7:0] s_rx_data;
```

```
==== Scoreboard Summary ====  
Total transaction: 1000  
PASS: 986  
FAIL: 14  
TEST FAILED: 14 mismatches detected!
```



```
constraint c_clk_div {  
    clk_div >= 8'h04;  
}
```

```
==== Scoreboard Summary ====  
Total transaction: 1000  
PASS: 1000  
FAIL: 0  
TEST PASSED: 1000 all matches detected!
```

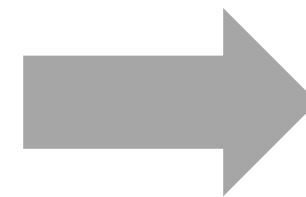
- Slave에서의 synchronizer로 인해 클럭의 지연이 발생하고 그 결과 clk\_div가 매우 작을 때 데이터가 밀리는 현상이 발생  
-> clk\_cnt 를 4이상으로 충분히 주어 지연을 방지

- Seq\_item에서 clk\_div를 4이상으로 설정하여 flipflop에 의한 클럭 지연을 해결

# Trouble shooting

```
[i2c_monitor] mon tx: OP: I2C_READ, MASTER -> tx:0x00 rx:0xff (ACK_OUT: 1), SLAVE -> tx:0xf9 rx:0x00  
[i2c_scoreboard] [READ FAIL] Slave TX: 0xf9 != Master RX: 0xff
```

```
if (is_read) begin  
    req.op = I2C_READ;  
    req.m_ack_in = 1'b1;  
    req.randomize();  
end else begin  
    req.op = I2C_WRITE;  
    req.randomize();  
end
```



```
[i2c_monitor] mon tx: OP: I2C_READ, MASTER -> tx:0x00 rx:0xf9 (ACK_OUT: 0), SLAVE -> tx:0xf9 rx:0x00  
[i2c_scoreboard] [READ PASS] Slave TX: 0xf9 == Master RX: 0xf9
```

```
if (is_read) begin  
    req.op = I2C_READ;  
    req.randomize();  
    req.m_ack_in = 1'b1;  
end else begin  
    req.op = I2C_WRITE;  
    req.randomize();  
end
```

- i2c\_sequence에서 m\_ack\_in 값을 1(NACK)로 할당한 직후 req.randomize()를 호출  
-> 할당된 값이 무작위 값으로 덮어씌워짐  
master가 0xff를 수신함

- req.randomize()를 먼저 수행한 후,  
req.m\_ack\_in = 1'b1을 덮어씌우도록 수정.

# Trouble shooting

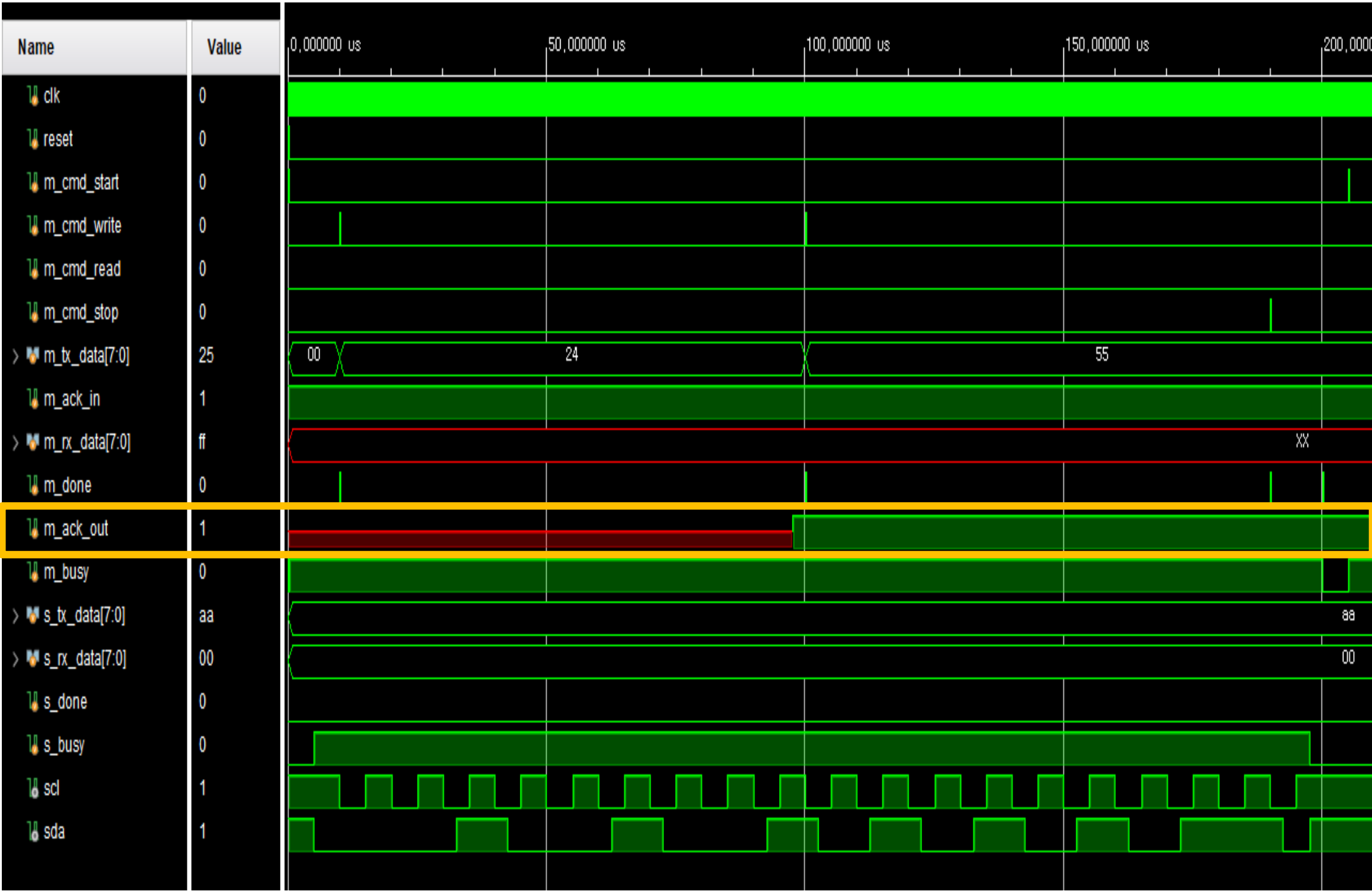
---

```
logic [2:0] bit_cnt;
```

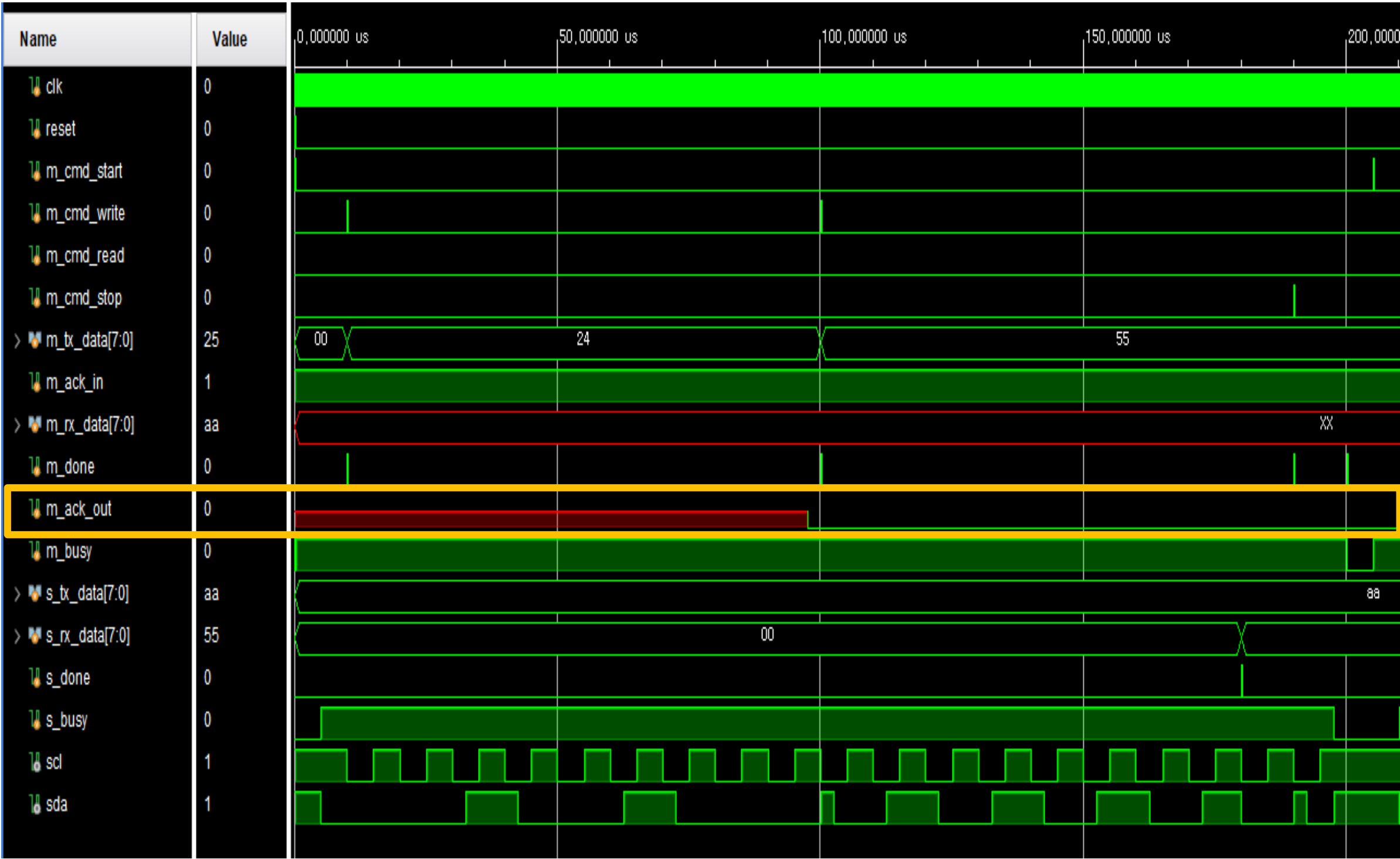
```
ADDR: begin
  if (scl_rise) begin
    rx_shift_reg <= {rx_shift_reg[6:0], sda_sync[1]};
  end
  if (scl_fall) begin
    if (bit_cnt == 7) begin
      if (rx_shift_reg[7:1] == SLA_ADDR) begin
        state <= ACK_ADDR;
        is_read <= rx_shift_reg[0];
        sda_r <= 1'b0;
      end else begin
        state <= IDLE;
      end
      bit_cnt <= 0;
    end else begin
      bit_cnt <= bit_cnt + 1;
    end
  end
end
end
```

- START는 비동기 인터럽트처럼 동작하기 때문에 ADDR 진입 직후 scl\_fall이 한 번 발생  
-> bit\_cnt == 8로 바꾸어줘야한다.
- bit\_cnt 가 [2:0]이므로 bit\_cnt == 8을 진입하지 못함.

# Trouble shooting



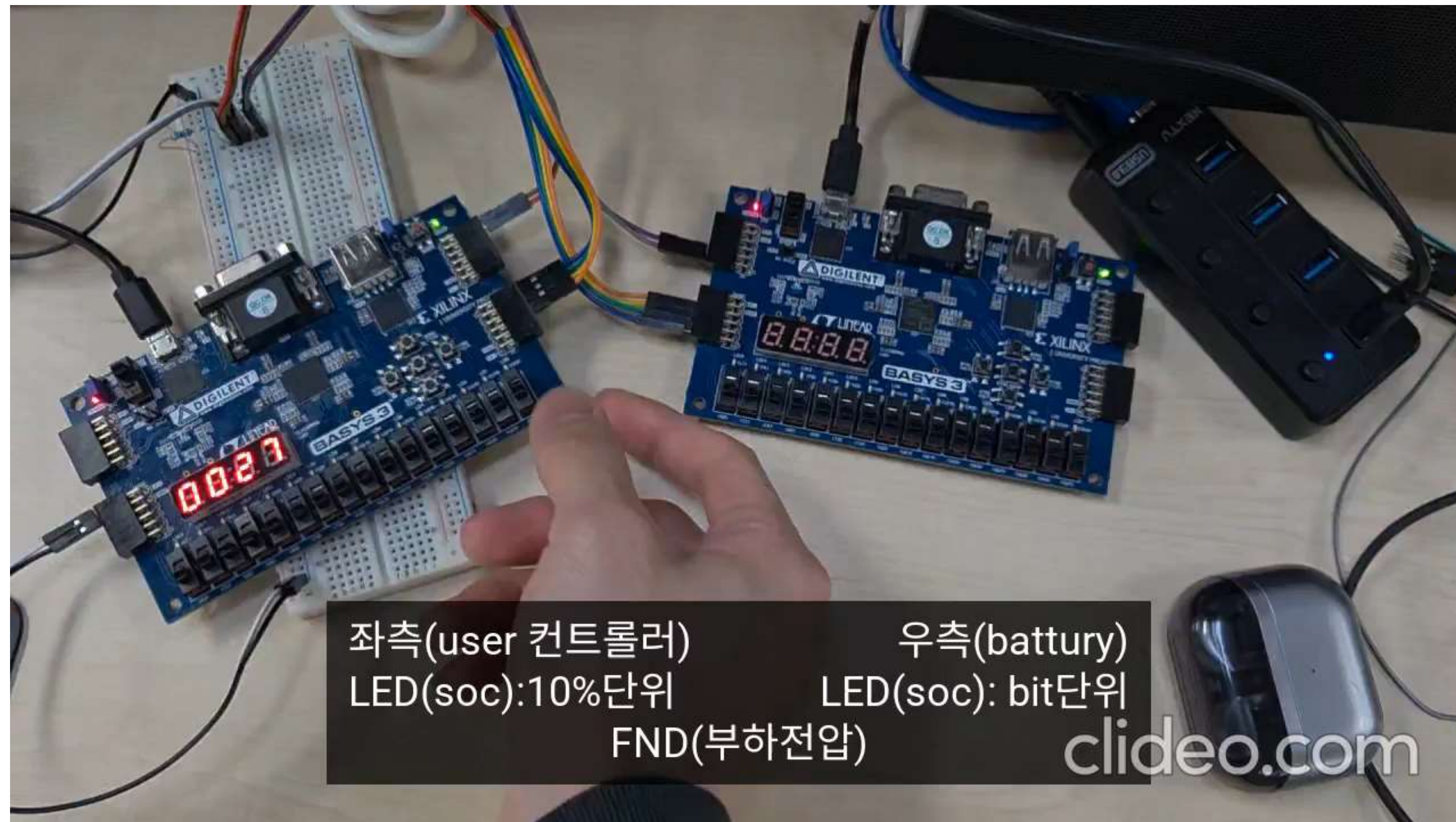
```
if (bit_cnt == 7) begin    logic [3:0] bit_cnt;
```



```
if (bit_cnt == 8) begin    logic [3:0] bit_cnt;
```

# | Demo Video

# Demo Video



# Q & A

대한상공회의소

최은수

온디바이스ai칩설계 1기